



Напишите / чтобы найти проекты



Помощь

Документы

Спонсоры

Войти

Зарегистрироваться

# pyqsp 0.1.1



Доступна новая  
версия (0.2.0)

```
pip install pyqsp==0.1.1
```



Выпущена: 30 апр. 2021  
г.

Генерация фазовых углов для алгоритмов квантовой обработки сигналов

## Навигация

☰ Описание  
проекта

🕒 История  
выпусков

📄 Загрузка  
файлов

Проверенные  
детали ✓

Эти сведения были  
[проверены PyPI](#)

Сопровождающие



ichuang

## Описание проекта

# Квантовая обработка сигналов

🔄 Run Tests no status

## Введение

[Квантовая обработка сигналов](#) — это набор квантовых алгоритмов, включающий в себя моделирование гамильтонианов, решение квантовых линейных систем, усиление амплитуды и т. д.

Квантовая обработка сигналов выполняет спектральное преобразование любого унитарного преобразования  $U$  при наличии доступа к вспомогательному кубиту, управляемой версии  $U$  и однокубитным поворотам на вспомогательном кубите. Сначала произвольная функция спектрального преобразования преобразуется в многочлен Лорана, а затем находится способ разложения многочлена Лорана на последовательность управляемых преобразований  $U$  и

## Непроверенные данные

Эти сведения *не*  
проверены PyPI

### Ссылки проекта

 [Homepage](#)

### Метаданные

- Лицензия: LICENSE.txt
- Автор: [L. Chuang](#) 



NVIDIA is a  
Visionary sponsor  
of the Python  
Software  
Foundation.

PSF Sponsor · Served  
ethically

Сообщить о  
вредоносном  
проекте

однокубитных поворотов (с определенными «фазовыми углами QSP») на вспомогательном кубите. Такие методы позволяют достичь оптимальной сложности вентилях для решения многих упомянутых выше задач квантовых алгоритмов. Решение поставленной задачи практически полностью определяется фазовыми углами QSP, используемыми в последовательности операций QSP. Таким образом, основная задача заключается в нахождении этих фазовых углов QSP для заданного многочлена Лорана.

Этот пакет Python генерирует фазовые углы квантовой обработки сигналов с помощью кода, основанного на двух разных методах. Метод «Лорана» использует [«Поиск углов для квантовой обработки сигналов с машинной точностью»](#) и расширяет исходный код для расчета фазовых углов квантовой обработки сигналов, представленный на <https://github.com/alibaba-edu/angle-sequence>. Метод «tf» использует TensorFlow + Keras и находит углы квантовой обработки сигналов с помощью оптимизации.

В литературе используются две модели квантовых логических элементов:  $W_x$ , где сигнал  $W(a)$  представляет собой поворот на  $X$ , а фазовые сдвиги квантовых логических элементов — поворот на  $Z$ , и  $W_z$ , где сигнал  $W(a)$  представляет собой поворот на  $Z$ , а фазовые сдвиги квантовых логических элементов — поворот на  $X$ .

В частности, в соглашении  $W_x$  последовательность операций QSP выглядит так:

$$U_x = e^{i\phi_0 Z} \sum_{k=1}^L W(a) e^{i\phi_k Z} \quad \text{where} \quad W(a) = \begin{bmatrix} a & i\sqrt{1-a^2} \\ i\sqrt{1-a^2} & a \end{bmatrix}$$

А в соглашении  $W_z$  последовательность операций QSP выглядит так:

$$U_z = e^{i\phi_0 X} \sum_{k=1}^L W(a) e^{i\phi_k X} \quad \text{where} \quad W(a) = \begin{bmatrix} a + i\sqrt{1-a^2} & 0 \\ 0 & a - i\sqrt{1-a^2} \end{bmatrix}$$

Они связаны преобразованием Адамара:

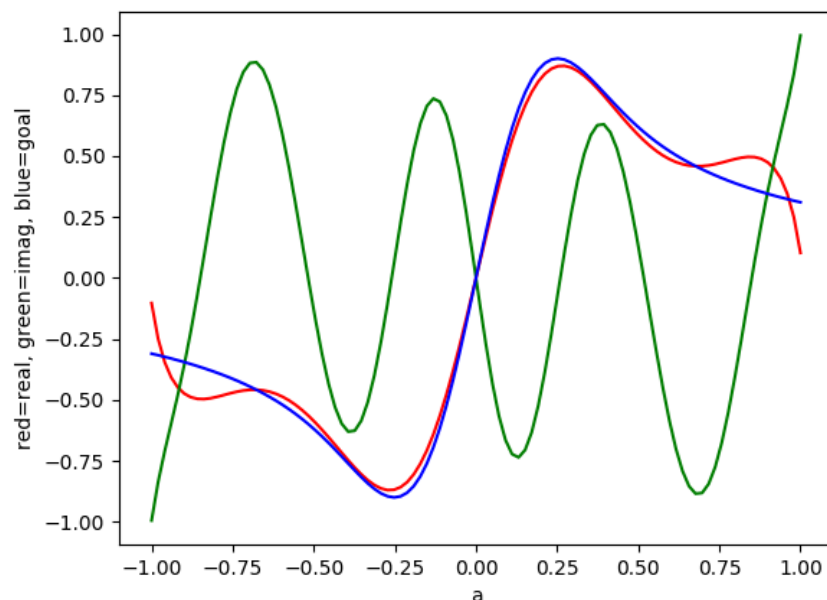
$$U_x = H U_z H$$

Соглашение  $W_z$  удобно и применяется в [полиномиальных формулировках квантовой теории поля](#), в то время как соглашение  $W_x$  является более традиционным, например в [квантовом сингулярном разложении](#).

Этот пакет может генерировать фазовые углы QSP для обоих соглашений (в то время как исходный код поддерживает только соглашение Wz). Сложность заключается в том, что если вам нужен определенный многочлен  $P_x(a) = \langle 0|U_x|0\rangle$  в соглашении Wx, то вы не можете просто использовать фазы, сгенерированные для этого многочлена в соглашении Wz. Вместо этого сначала нужно вычислить  $Q_x(a)$ , соответствующий  $P_x(a)$ , чтобы получить полный  $U_x$ . В результате получается  $P_z(a) = \langle 0|U_z|0\rangle = P_x(a) + Q_x(a)$ . Вычисление QSP-фаз для  $P_z(a)$  в соответствии с соглашением Wz дает искомые QSP-фазы для  $P_x(a)$  в соответствии с соглашением Wx. Обратите внимание, что это не совсем удовлетворительное решение, поскольку между  $P_x$  и  $Q_x$  остается неопределенная фаза, а  $Q_x$  может содержать ошибки из-за чувствительности процесса завершения к корням задействованных многочленов, но эти проблемы, скорее всего, можно решить.

## Примеры

Этот пакет также позволяет построить функцию отклика QSP и может быть запущен из командной строки. В приведенном ниже примере синяя линия показывает целевую идеальную полиномиальную функцию отклика QSP  $P_x(a)$ ; красная линия показывает действительную часть отклика, достигаемого с помощью фаз QSP, а зеленая линия — мнимую часть отклика QSP при  $L=20$ .



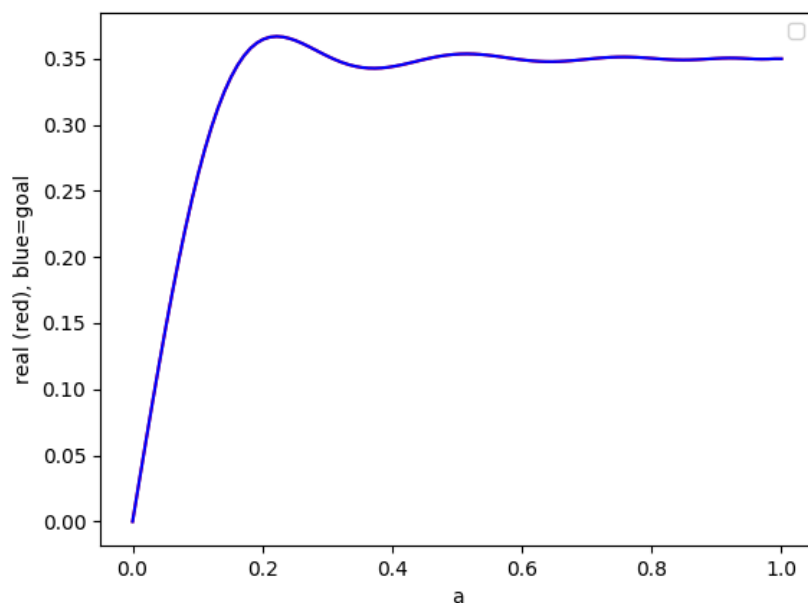
Это было сделано с помощью команды `pyqsp --plot invert`, которая выдала следующий текстовый вывод:

```
b=20, j0=10
[PolyOneOverX] minimum [-2.90002589] is at [-0.25174082]
Laurent P poly 0.0002108924030879319 * w ^ (-21) + -0.0002108924030879319 * w ^ (-20) + ...
Laurent Q poly -2.784691352688532e-05 * w ^ (-21) + -3.169381961797737e-05 * w ^ (-20) + ...
Laurent Pprime poly 0.00018304548956104657 * w ^ (-21) + -0.00018304548956104657 * w ^ (-20) + ...
[QuantumSignalProcessingWxPhases] Error in reconstructing QSP angles = [0.11177647198529908, 0.321286291382733, ...]
```

Функция `sign` также полезна для аппроксимации, например, при незаметном усилении амплитуды. Запуск

```
pyqsp --plot-positive-only --plot --polyargs=19,10 --plot
```

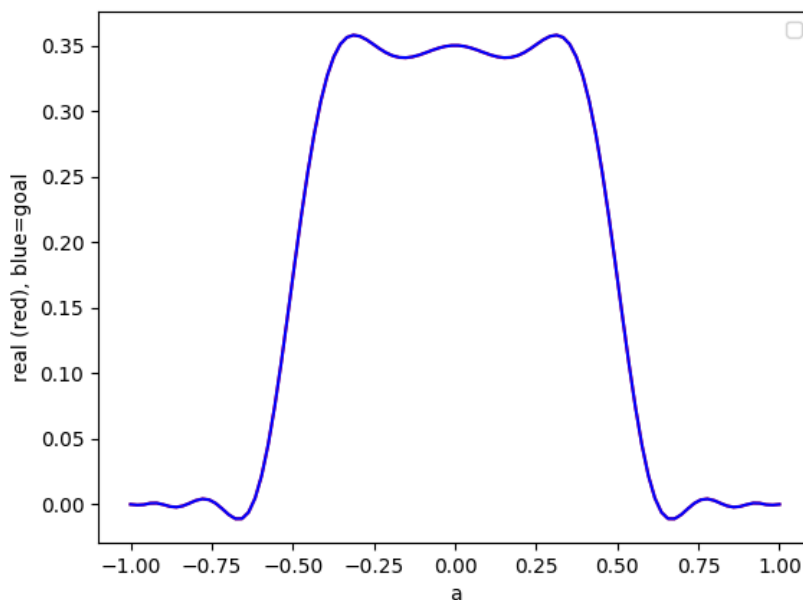
Выводит фазовые углы QSP для полиномиальной аппроксимации 19-й степени с использованием функции ошибки  $\text{карра} * a$ , где  $\text{карра}$  равно 10, с функцией отклика, показанной на этом графике:



Пороговая функция полезна, например, для различения собственных и сингулярных значений. Запуск

```
pyqsp --plot-real-only --plot --polyargs=20,20 --polyn
```

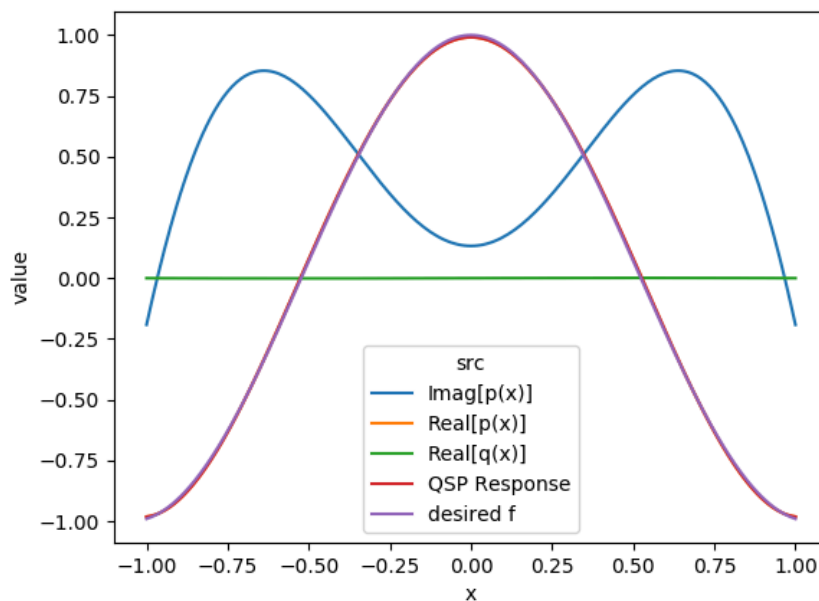
Выводит фазовые углы QSP для полиномиальной аппроксимации степени 20 с использованием двух функций погрешности карра 20 и функции отклика, как показано на этом графике:



Функции синуса и косинуса полезны, например, для гамильтонова моделирования. Выполнение:

```
pyqsp --plot --func "np.cos(3*x)" --polydeg 6 --plot-qsp
```

вычисляет фазовые углы QSP для полиномиальной аппроксимации  $\cos(3*x)$  степени 6 и строит график:



В этом примере также показано, как можно задать произвольную функцию (с помощью выражения `numpy`) в виде строки и подогнать ее под полином произвольного порядка (он может быть четным или нечетным, в зависимости от функции) с помощью оптимизации через TensorFlow и модели Keras. В примере также показан альтернативный стиль построения графика с использованием флага `--plot-qsp-model`.

## Разработка кода

- `angle_sequence.py` — основной модуль алгоритма.
- `LPoly.py` определяет два класса `LPoly` и `LAlg`, представляющих полиномы Лорана и элементы алгебры Лоу соответственно.
- `completion.py` описывает алгоритм завершения: для заданного полинома Лорана  $F(\tilde{w})$  найти его аналог  $G(\tilde{w})$  такой, что  $F(\tilde{w}) + G(\tilde{w}) * iX$  является унитарным элементом.
- `decomposition.py` описывает алгоритм половинного деления: для заданного элемента унитарной алгебры четности  $V(\tilde{w})$  разложите его на уникальное произведение поворотов нулевой степени  $\exp[i\theta X]$  и одночленов первой степени  $w$ .
- `ham_sim.py` показывает пример того, как можно найти последовательность углов для гамильтонова моделирования.
- `response.py` вычисляет функции отклика QSP и строит графики
- `poly.py` предоставляет несколько вспомогательных полиномов, в частности аппроксимацию  $1/a$  с помощью линейной комбинации полиномов Чебышёва
- `main.py` является основной точкой входа для использования в командной строке
- `qsp_model` является подмодулем, обеспечивающим вычисление фазовых углов QSP с помощью tensorflow + keras

Код структурирован таким образом, что tensorflow по умолчанию не импортируется, поэтому пакет можно запустить без установки tensorflow. Если используется `qsp_model`, то tensorflow необходим.

## Модульные тесты

Также предоставляется набор модульных тестов. Запустите их с помощью `python setup.py test`

Код `qsp_model` зависит от наличия установленного TensorFlow, а модульные тесты для этого кода выполняются довольно долго, поэтому по умолчанию они не запускаются. Чтобы включить модульные тесты для этого кода, сначала выполните `export PYQSP_TEST_QSP_MODELS=1`

Модульные тесты `qsp_model` кода можно запускать по отдельности с помощью `python setup.py test -s pyqsp.test.test_qsp_models`

## программного использования

Чтобы найти последовательность углов QSP, соответствующую реальному многочлену Лорана  $A(\tilde{w}) = \sum_{i=-n}^n a_i \tilde{w}^i$ , просто запустите:

```
from pyqsp.angle_sequence import QuantumSignalProcessing
ang_seq = QuantumSignalProcessingPhases([a_{-n}, a_{-n-1}, ..., a_n])
print(ang_seq)
```

To find the QSP angle sequence corresponding to a real (non-Laurent) polynomial  $A(x) = \sum_{i=0}^n a_i x^i$ , simply run:

```
from pyqsp.angle_sequence import QuantumSignalProcessing
ang_seq = QuantumSignalProcessingPhases([a_{0}, a_{1}, ..., a_n])
print(ang_seq)
```

By default, `QuantumSignalProcessingPhases` uses the "laurent" method, which is typically quite fast, but can become unstable at very high orders of polynomials, due to numerical roundoff errors, and the need for some randomization in completing the polynomials.

`QuantumSignalProcessingPhases` can also be instructed to use the "tf" method, which employs tensorflow with a keras model, to find QSP phase angles using optimization. This stably finds very high-quality solutions, but can be quite slow, particularly compared with the "laurent" method. Do this, for example, using:

```
ang_seq = QuantumSignalProcessingPhases(poly, signal_op
```

Note that with the "tf" method, only the "Wx" signal\_operator model is supported. With this method, the polynomial can be a numpy Polynomial instance, or an instance of

```
pyqsp.poly.StringPolynomial, e.g.
```

```
poly = StringPolynomial("np.cos(3*x)", 6)
ang_seq = QuantumSignalProcessingPhases(poly, method="tf")
```

You can also plot the response given by a given QSP angle sequence, e.g. using:

```
pyqsp.response.PlotQSPResponse(ang_seq, target=poly, s
```

## Command line usage

```
usage: pyqsp [-h] [-v] [-o OUTPUT] [--signal_operator S]
             [--polydeg POLYDEG] [--tau TAU] [--epsilon EPSILON]
             [--plot-magnitude] [--plot-real-only] [--color COLOR]
             [--tolerance TOLERANCE] [--method METHOD]
             cmd
```

```
usage: pyqsp [options] cmd
```

Version: 0.1.0

Commands:

```
poly2angles - compute QSP phase angles for the specified polynomial
hamsim      - compute QSP phase angles for Hamiltonian simulation
invert      - compute QSP phase angles for matrix inversion
angles      - generate QSP phase angles for the specified polynomial
poly        - generate QSP phase angles for the specified polynomial
polyfunc    - generate QSP phase angles for the specified polynomial
```

Examples:

```
pyqsp --poly=-1,0,2 poly2angles
pyqsp --poly=-1,0,2 --plot poly2angles
pyqsp --signal_operator=Wz --poly=0,0,0,1 --plot magnitude
pyqsp --plot --tau 10 hamsim
pyqsp --plot --tolerance=0.01 --seqargs 3 invert
```



```

pyqsp --plot-npts=4000 --plot-positive-only --plot-
pyqsp --plot-npts=100 --plot-magnitude --plot --sec
pyqsp --plot-npts=100 --plot-positive-only --plot -
pyqsp --plot-real-only --plot --polyargs=20,20 --po
pyqsp --plot-positive-only --plot --polyargs=19,10
pyqsp --plot-positive-only --plot-real-only --plot
pyqsp --plot-positive-only --plot-real-only --plot
pyqsp --plot-positive-only --plot --polyargs=19,10
pyqsp --plot --func "np.cos(3*x)" --polydeg 6 polyt
pyqsp --plot --func "np.cos(3*x)" --polydeg 6 --plo
pyqsp --plot-positive-only --plot-real-only --plot

```

#### positional arguments:

|     |         |
|-----|---------|
| cmd | command |
|-----|---------|

#### optional arguments:

|                                   |                                 |
|-----------------------------------|---------------------------------|
| -h, --help                        | show this help message and exit |
| -v, --verbose                     | increase output verbosity (add  |
| -o OUTPUT, --output OUTPUT        | output filename                 |
| --signal_operator SIGNAL_OPERATOR | QSP sequence signal_operator, e |
| --plot                            | generate QSP response plot      |
| --hide-plot                       | do not show plot (but it may be |
| --return-angles                   | return QSP phase angles to call |
| --poly POLY                       | comma delimited list of floatin |
| --func FUNC                       | for tf method, numpy expression |
| --polydeg POLYDEG                 | for tf method, degree of polyno |
| --tau TAU                         | time value for Hamiltonian simu |
| --epsilon EPSILON                 | parameter for polynomial approx |
| --seqname SEQNAME                 | name of QSP phase angle sequenc |
| --seqargs SEQARGS                 | arguments to the phase angles g |
| --polynome POLYNOME               | name of polynomial generate us  |
| --polyargs POLYARGS               | arguments to the polynomial ger |
| --plot-magnitude                  | when plotting only show magnitu |
| --plot-real-only                  | when plotting only real compone |
| --output-json                     | output QSP phase angles in JSON |
| --plot-positive-only              | when plotting only a-values (x- |
| --plot-tight-y                    | when plotting scale y-axis tigh |
| --plot-npts PLOT_NPTS             | number of points to use in plot |
| --tolerance TOLERANCE             | error tolerance for phase angle |
| --method METHOD                   | method to use for qsp phase ang |
| --plot-qsp-model                  | show qsp_model version of respo |

## History

- v0.0.3: initial version, with phase angle generation entirely done using <https://arxiv.org/abs/2003.02831>
- v0.1.0: added generation of phase angles using optimization via tensorflow (qsp\_model code by Jordan Docter and Zane Rossi)
- v0.1.1: add tf unit tests to test\_main; readme updates



## Помощь

Установка пакетов [↗](#)  
Загрузка пакетов [↗](#)  
Руководство пользователя [↗](#)  
Удержание названия  
проекта [↗](#)  
Часто задаваемые вопросы

## О PyPI

Блог PyPI [↗](#)  
Панель мониторинга  
инфраструктуры [↗](#)  
Статистика  
Логотипы и товарные знаки  
Наши спонсоры

## Внесение вклада в PyPI

Программные ошибки и  
обратная связь  
Внесение вклада на GitHub'е [↗](#)  
Перевод PyPI [↗](#)  
Спонсор PyPI  
Вклад в разработку [↗](#)

## Использование PyPI

Условия и положения [↗](#)  
Сообщить о проблеме  
безопасности  
Нормы поведения [↗](#)  
Политика  
конфиденциальности [↗](#)  
Политика допустимого  
использования [↗](#)

Статус: All Systems Operational [↗](#)

Разработано и поддерживается сообществом Python'а для сообщества Python'а.  
Пожертвуйте сегодня!

PyPI, "Python Package Index" и логотипы блоков являются зарегистрированными  
товарными знаками Python Software Foundation [↗](#).

Переключиться на настольную версию

English

español

français

日本語

português (Brasil)

українська

Ελληνικά

Deutsch

中文 (简体)

中文 (繁體)

русский

עברית

Esperanto

한국어

AWS

Cloud computing  
and Security  
Sponsor

Datadog

Monitoring

Depot

Continuous  
Integration

Fastly

CDN

Google

Download Analytics

Pingdom

Monitoring

Sentry

Error logging

StatusPage

Status page